Import Statements

In [ ]:
```
import custom_cmap
import os
import sys
from PIL import Image
from IPython.display import display
import matplotlib.pyplot as plt
import matplotlib as mpl
import pandas as pd
from moria import reduce

from pathlib import Path
from astropy.visualization import PercentileInterval

from astropy.io import fits
from astropy.visualization import LogStretch, ImageNormalize
import plotly.express as px
import numpy as np
from astropy.visualization import ImageNormalize, AsinhStretch, SqrtStretch,

plt.rcParams.update({'font.size':25})
plt.rc('text', usetex=True)
%config InlineBackend.figure_format = 'retina'
%load_ext autoreload
%autoreload 2
```

Understanding the main directory.

The data directory should be organized as follows. You can look at the sample data folder to understand the directory strucutre you need.

00.DATA
01.XYM
02.CMD
03.LOC_TRANS
04.PSF_EXTRACT
05.COORD_TRANS(OPTIONAL)
06.FIT
07.CALIBRATION
All the necessary scripts are included in the sample data directory. The simplest way to run MORIA on your target is to copy the entirety of the "data" folder here, put your exposures in "data/00.DATA", then beginning with the demo.

Using the bright nearby reference stars identified in the CMD, more local transformations are derived for each exposure. Pixel data surrounding the target star and the selected

PSF-star candidates are extracted from each exposure and accurately transformed into a reference frame corrected for distortion. The extracted pixel list contains the flux location of each pixel relative to the star centers and is fundamental for building the PSF model.

For each filter, using the stars selected to have similar magnitudes and colors to the target star, MORIA builds a PSF model. Candidate PSF stars are evaluated using residual images from PSF subtraction and stars that show significant residuals due to blending or large Poisson noise are excluded in a second iteration used for PSF modeling. The final output is to derive a high-resolution PSF model for each filter.

After you finish running this notebook, you will be able to use the "_flc" exposures to generate an output image stack in the F814W and F606W HST filters.

NOTE: chmod +x program.src is a useful command to use whenever permissions are denied for a script.

In [ ]:
```
directory = os.getcwd()
#code_directory = os.path.abspath("..")
#sys.path.append(code_directory)
```

Step 0

We assume you ran output_stacks.ipynb and then cmd_diagram.ipynb (in that order)

Step 1

Generate a more local transformation from each frame into the reference frame, then to use this transformation to extract the pixels from each exposure and accurately transform their locations into the reference frame so that we can use them to solve for a PSF and then use this PSF to model the target star.

In [3]:
```
reduce.loc_trans(directory)
```

```
No such comm target registered: jupyter.widget.control
No such comm: ea73f885-dc79-411b-a146-23596162c70a
```

Step 2

Generate a local PSF for each filter, using the stars that are similar in magnitude and color to the target star. The magnitude similarity is thought to be important because the CTE losses are expected to make PSF shapes magnitude dependent.

Sometimes this step might not work. This means the numbre of sim stars you have is too high. Reduce the number of sim stars in one of the earlier steps.

In [4]:
```
reduce.extract_psf_1(directory)
```

```
Enter the number of images for the F814W filter:  24
Enter the number of images for the F606W filter:  24
```

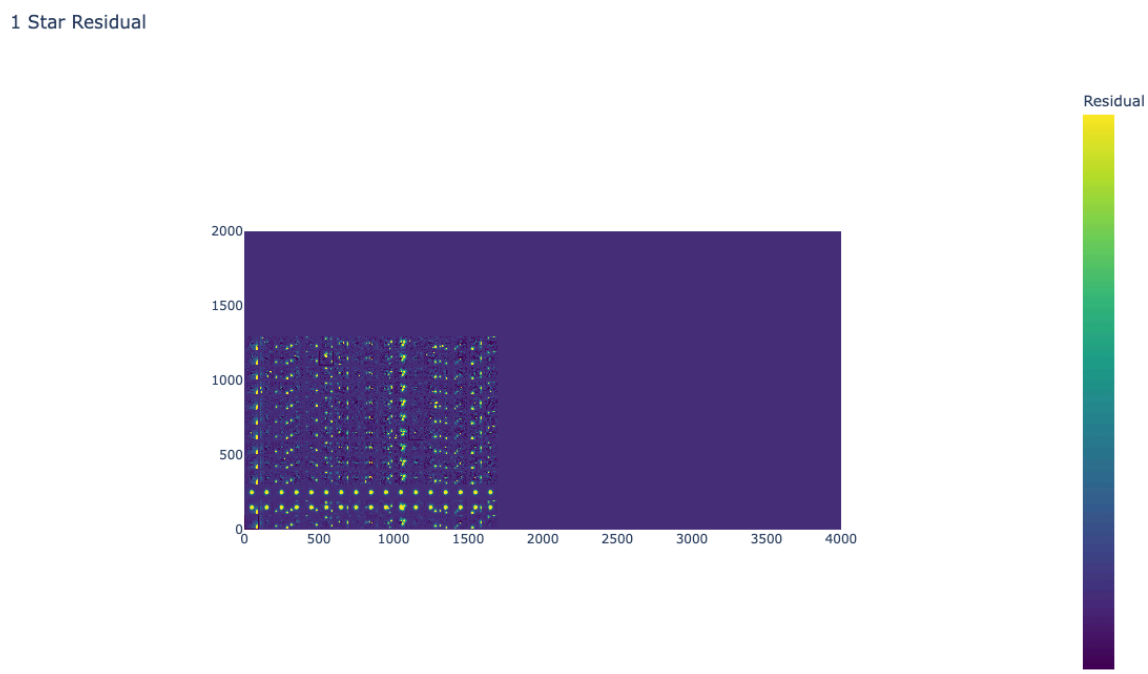
Step 3

Loook at the fit file generated above. Then look at the bottom panel. Go through it to select "good" stars. Ensure that the residuals are smooth

In [5]:
```
fit_file = Path(directory).resolve()/f"04.EXTRACT_PSF/F814W/show_str_simst1.

hdul = fits.open(fit_file)
data = hdul[-1].data
hdul.close()
interval = PercentileInterval(99)
scaled = interval(data)

fig = px.imshow(
    scaled,
    origin='lower',
    color_continuous_scale='viridis',
    title="1 Star Residual",
    aspect='equal'
)

fig.update_layout(
    width=750,
    height=750,
    coloraxis_colorbar=dict(title="Residual", tickvals=[]),
)
fig.show()
```

```
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
 [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CRPIX1  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CRPIX2  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CRVAL1  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CRVAL2  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CTYPE1  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CTYPE2  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CD1_1   =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CD1_2   =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CD2_1   =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CD2_2   =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
ORIENTAT=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
PA_APER =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
PA_V3   =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
DATE-OBS=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
TIME-OBS=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
EXPTIME =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
ROOTNAME=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
```

```
TARGNAME=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
RA_TARG =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
DEC_TARG=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
PROPOSID=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
FILTER1 =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
FILTER2 =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CCDGAIN =  [astropy.io.fits.card]
```



Step 4: Input the PSF stars you like from your selection above

Input the panel number for the PSF stars you like as a list below:

In [6]:
```
good_panels = [3, 4, 11, 14, 15, 16]
```

In [ ]:
```
#Change number_of_sim_stars to the number of sim stars you have
good_psf = np.zeros(number_of_sim_stars=30, dtype = int)
```

```
for index in good_panels:
    good_psf[index] = 1
```

Step 5: Now run PSF Extract again.

This will create a good PSF for your target.

In [8]:
```
reduce.extract_psf_2(good_psf, directory)
```

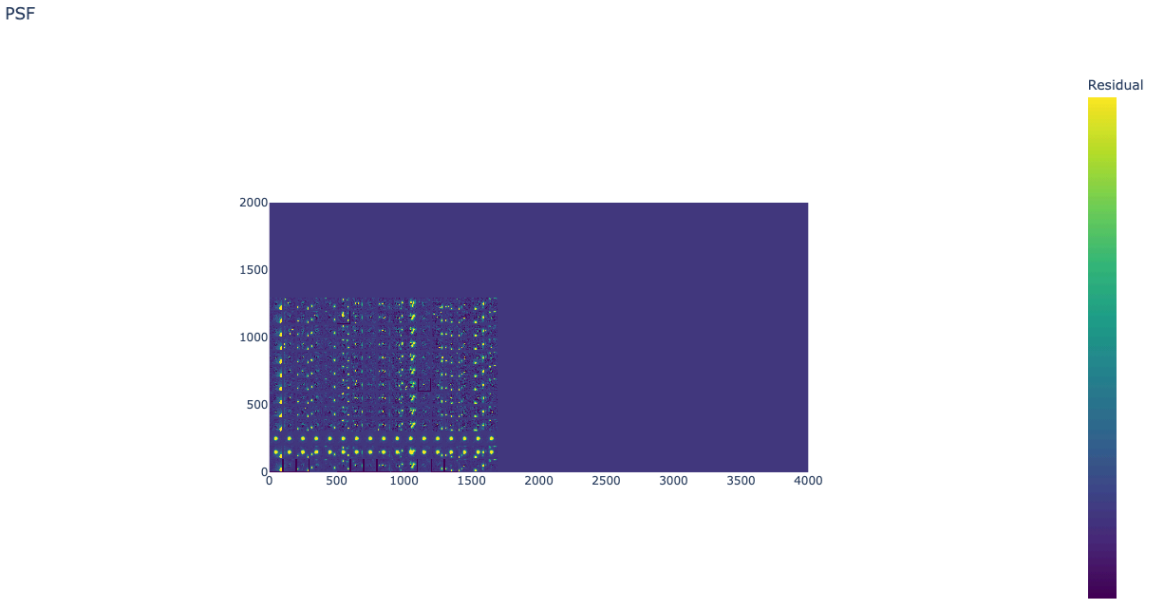
```
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
 [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CRPIX1  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CRPIX2  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CRVAL1  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CRVAL2  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CTYPE1  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CTYPE2  =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CD1_1   =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CD1_2   =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CD2_1   =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CD2_2   =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
ORIENTAT=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
PA_APER =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
PA_V3   =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
DATE-OBS=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
TIME-OBS=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
EXPTIME =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
ROOTNAME=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
```

```
TARGNAME=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
RA_TARG =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
DEC_TARG=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
PROPOSID=  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
FILTER1 =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
FILTER2 =  [astropy.io.fits.card]
WARNING: The following header keyword is invalid or follows an unrecognized
non-standard convention:
CCDGAIN =  [astropy.io.fits.card]
```

Step 11

Loook at the PSF generated after your good selection of stars.

In [9]:
```
fit_file = Path(directory).resolve()/f"04.EXTRACT_PSF/F814W/show_str_simst2.
#fit_file = Path(directory).resolve()/f"04.EXTRACT_PSF/F814W/psfout_simst.fi

hdul = fits.open(fit_file)
data = hdul[-1].data
hdul.close()
interval = PercentileInterval(99)
scaled = interval(data)

fig = px.imshow(
    scaled,
    origin='lower',
    color_continuous_scale='viridis',
    title="PSF",
    aspect='equal'
)

fig.update_layout(
    width=750,
    height=750,
    coloraxis_colorbar=dict(title="Residual", tickvals=[]),
)
fig.show()
```


PSF
Residual
2000
1500
1000
500
0
0
500
1000
1500
2000
2500
3000
3500
4000

In [ ]: